

CHAPTER 21

Regularization for Linear Models

Regularization is the technique of adding a parameter, λ , to the loss function of a learning algorithm to improve its ability to generalize to new examples by reducing overfitting. The role of the extra regularization parameter is to shrink or to minimize the measure of the weights (or parameters) of the other features in the model.

Regularization is applied to linear models such as polynomial linear regression and logistic regression which are susceptible to overfitting when high-order polynomial features are added to the set of features.

How Does Regularization Work

During model building, the regularization parameter λ is calibrated to determine how much the magnitude of other features in the model is adjusted when training the model. The higher the value of the regularization, the more the magnitude of the feature weights is reduced.

If the regularization parameter is set too close to zero, it reduces the regularization effect on the feature weights of the model. At zero, the penalty the regularization term imposes is virtually non-existent, and the model is as if the regularization term was never present.

Effects of Regularization on Bias vs. Variance

The higher the value of λ (i.e., the regularization parameter), the more restricted the coefficients (or weights) of the cost function. Hence, if the value of λ is high, the model can result in a learning bias (i.e., it underfits the dataset).

However, if the value of λ approaches zero, the regularization parameter has negligible effects on the model, hence resulting in overfitting the model. Regularization is an important technique and should be used when injecting polynomial features into linear or logistic regression classifiers to learn non-linear relationships.

Applying Regularization to Models with Scikit-learn

The technique of adding a penalty to restrain the values of the parameters of the model is also known as Ridge regression or Tikhonov regularization. In this section we will build a linear and logistic regression model with regularization.

Linear Regression with Regularization

This code block is similar to the polynomial linear regression example in Chapter 19. The model will predict house prices from the Boston house-prices dataset. However, this model includes regularization.

```
# import packages
from sklearn.linear_model import Ridge
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from math import sqrt
from sklearn.preprocessing import PolynomialFeatures

# load dataset
data = datasets.load_boston()

# separate features and target
X = data.data
y = data.target

# create polynomial features
polynomial_features = PolynomialFeatures(2)
X_higher_order = polynomial_features.fit_transform(X)
```

```
# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X_higher_order, y,
                                                    shuffle=True)

# create the model. The parameter alpha represents the regularization
# magnitude
linear_reg = Ridge(alpha=1.0)

# fit the model on the training set
linear_reg.fit(X_train, y_train)

# make predictions on the test set
predictions = linear_reg.predict(X_test)

# evaluate the model performance using the root mean square error metric
print("Root mean squared error (RMSE): %.2f" % sqrt(mean_squared_error(y_
test, predictions)))
```

'Output':

Root mean squared error (RMSE): 3.74

Take note of the following:

- The method `Ridge(alpha=1.0)` initializes a linear regression model with regularization, where the attribute 'alpha' controls the magnitude of the regularization parameter.

Logistic Regression with Regularization

This code block here is also similar to the example in Chapter [20](#) on logistic regression. The model will predict the three species of flowers from the Iris dataset. The addition to this code segment is the inclusion of a regularization term to the logistic model using the 'RidgeClassifier' package.

```
# import packages
from sklearn.linear_model import RidgeClassifier
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```

# load dataset
data = datasets.load_iris()

# separate features and target
X = data.data
y = data.target

# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True)

# create the logistic regression model
logistic_reg = RidgeClassifier()

# fit the model on the training set
logistic_reg.fit(X_train, y_train)

# make predictions on the test set
predictions = logistic_reg.predict(X_test)

# evaluate the model performance using accuracy metric
print("Accuracy: %.2f" % accuracy_score(y_test, predictions))

'Output':
Accuracy: 0.76

```

In the preceding code block, logistic regression with regularization is implemented by the method ‘`RidgeClassifier()`’. The reduced accuracy observed in this example when regularization is applied to logistic regression is because the algorithm is restricting the values of the model parameters to prevent high variance on a dataset that is fairly simplistic and already has high accuracy on test samples without regularization.

This chapter discusses the role of regularization in linear models like linear and logistic regression. Other forms of regularization exist for other model types such as early stopping for neural networks (to be discussed later in Chapter 34). Regularization is an important technique when designing machine learning models. The next chapter will discuss and implement another important machine learning algorithm known as support vector machines.